CSE 5525: Assignment 3

Qifan Wen

Resource	VM 1 (Parts 1 & 2)	VM 2 (Part 3)
OS	Ubuntu 22.04 LTS (Linux 5.4, x86_64)	Ubuntu 24.04 LTS (Linux 6.8, x86_64)
GPU	NVIDIA A40 (46 GB VRAM)	NVIDIA RTX 5090 (32 GB VRAM)
GPU driver	535.129.03	590.48.01
CPU	AMD EPYC 7H12 64-Core (128 threads)	AMD EPYC 9654 96-Core (384 threads)
RAM	1 TB	755 GB
Storage	50 GB overlay (NVMe)	60 GB overlay (NVMe)
Software	Python 3.10, PyTorch 2.10, Transformers 5.2	Python 3.12, PyTorch 2.10, Transformers 5.3
CUDA / cuDNN	12.2 / 9.1	12.8 / 9.1
Batch sizing	Automatic per-config tuning to maximize VRAM utilization	

Table 1: Hardware and software environment.

1 Data Statistics and Processing (8pt)

Table 2 reports the raw dataset statistics before any model-specific preprocessing. All token counts use the T5 tokenizer (`T5TokenizerFast` from `google-t5/t5-small`). Vocabulary sizes are computed over unique whitespace-delimited words.

Statistics Name	Train	Dev
Number of examples	4225	466
Mean sentence length	18.1	18.1
Mean SQL query length	217.4	211.1
Vocabulary size (natural language)	868	444
Vocabulary size (SQL)	644	393

Table 2: Data statistics before pre-processing (T5 tokenizer).

Table 3 shows per-model statistics after preprocessing. The T5 fine-tuned model prepends table names from the flight database schema to each natural language input (e.g., "tables: city, airport, flight, ..."), adding approximately 93 tokens per example. The T5 from-scratch model receives raw NL input without any schema augmentation. Both models restrict the decoder output vocabulary to 598 SQL-relevant tokens (SQL keywords, table/column names, operators, and literals extracted from the training data).

Statistics Name	Train	Dev
T5 Fine-Tune		
Mean encoder input length	111.1	111.1
Mean decoder target length	217.4	211.1
Encoder token vocab size	815	493
Decoder token vocab size	556 (598 restricted)	396 (598 restricted)
T5 From Scratch		
Mean encoder input length	18.1	18.1
Mean decoder target length	217.4	211.1
Encoder token vocab size	792	466
Decoder token vocab size	556 (598 restricted)	396 (598 restricted)

Table 3: Data statistics after pre-processing.

2 T5 Fine-tuning and Training From Scratch (8pt)

2.1 Supervised Fine-tuning (SFT)

Table 4 describes the data processing, tokenization, architecture, and hyperparameters for the fine-tuned and from-scratch T5 models.

Design choice	Description
Data processing	**Fine-tuned:** schema table names prepended to NL input. **From scratch:** raw NL input, no schema augmentation.
Tokenization	`T5TokenizerFast` from `google-t5/t5-small`; restricted decoder vocabulary (598 SQL tokens). Shared across both settings.
Architecture	**Fine-tuned:** T5-small (60M params) with restricted vocab head; also explored LoRA and MLP heads. **From scratch:** same T5-small architecture from random initialization.
Varying parameters	**Fine-tuned:** LR, batch size, dropout, schema, encoder freezing, adaptation method, max epochs, warmup, eval frequency, patience, wall-clock budget. **From scratch:** LR, batch size, dropout, label smoothing, max epochs, warmup, eval frequency, patience, wall-clock budget.
Constant parameters	Optimizer, scheduler, gradient clipping, loss function, beam search decoding, max generation length, seed, restricted decoder vocabulary.

Table 4: Design choices for T5 fine-tuned and from-scratch models.

2.2 Direct Preference Optimization (DPO)

Table 5 describes the DPO post-training pipeline applied on top of the best SFT model: the T5-base "Base sweep best" checkpoint (`google-t5/t5-base`, 222M params; see Table 12, F1 = 85.96%, SQL EM = 73.39%). DPO *(Rafailov et al., 2023)* optimizes the policy to prefer "chosen" over "rejected" completions without training a separate reward model, using the SFT checkpoint as both initialization and frozen reference. Results are reported in Section 4.

2.3 RL Fine-Tuning (PPO, GRPO, CISPO)

- **PPO** *(Schulman et al., 2017)*: Clipped surrogate objective with a learned value head (2-layer MLP on mean-pooled encoder hidden states, zero-initialized) as baseline; entropy bonus ($\alpha = 0.01$) for exploration; KL penalty ($\beta = 0.02$).
- **GRPO** *(Shao et al., 2024)*: PPO-style clipped surrogate with group-relative advantage normalization (no critic network). Stronger KL penalty ($\beta = 0.05$). Trained on larger subset (500 queries/iteration).
- **CISPO** *(MiniMax, 2025)*: Detached clamped importance sampling weights that preserve gradient flow for all tokens regardless of ratio magnitude. No KL penalty ($\beta = 0$).

Design choice	Description
Base model	T5-base "Base sweep best" (222M params, full fine-tune, restricted vocab, schema augmentation, LR = 6.19e-5, label smoothing = 0.15). Used as both policy initialization and frozen reference.
Preference data	9,801 triplets (NL, chosen SQL, rejected SQL). **Phase A:** model-generated negatives — up to 10 candidates per query via diverse beam search, classified by execution correctness against `flight_database.db`. **Phase B:** execution-aware perturbations of gold SQL (city swaps, date changes, operator flips), verified to produce different record sets. In-memory SQLite for fast evaluation (46K+ queries in ∼20 min).
Loss	$\mathcal{L}_{\text{DPO}} = -\log\sigma(\beta\,[\log\pi_\theta(y_w\|x) - \log\pi_{\text{ref}}(y_w\|x) - \log\pi_\theta(y_l\|x) + \log\pi_{\text{ref}}(y_l\|x)])$. Log probs computed via restricted SQL vocab (598 tokens); OOV tokens masked to zero.
Varying parameters	$\beta \in \{0.05, 0.1, 0.2, 0.3, 0.5\}$, LR $\in [10^{-7}, 5\times10^{-5}]$, weight decay, gradient clipping, architecture (full fine-tune vs. LoRA r=16 on q/v or q/k/v/o). W&B random sweep.
Constant parameters	Optimizer (AdamW), beam search decoding (num_beams = 4), restricted decoder vocabulary, schema augmentation, eval every epoch, patience = 5 eval cycles.

Table 5: Design choices for DPO post-training on T5-base.

Design choice	Description
Base model	T5-base "Base sweep best" (222M params, F1 = 85.96%). Used as frozen reference for KL penalty; LoRA adapter applied on top as trainable policy.
Adapter	LoRA $r = 32$, $\alpha = 64$, targeting q/k/v/o projections (6.86M trainable params, 3.08% of total).
Sampling	$G = 4$ completions per query via constrained decoding (temperature = 0.7, top-k = 50, max_new_tokens = 512, restricted 598-token SQL vocab). Sub-batch generation (chunk size 16) to bound peak VRAM during sequence generation.
Reward	Execution-based graded: +1.0 (exact record match), +0.5 (partial overlap), −0.5 (no record overlap), −1.0 (SQL execution error). Threaded reward computation (128 workers) with in-memory SQLite.
Algorithms	**PPO:** clipped surrogate + learned value head (MLP on mean-pooled encoder states, zero-initialized). **GRPO:** group-relative advantages, no critic, KL penalty. **CISPO:** detached clamped IS weights preserving gradient flow.
Varying parameters	Algorithm (PPO/GRPO/CISPO), train subset size (200–500 queries/iteration), KL penalty $\beta \in \{0, 0.02, 0.05\}$.
Constant parameters	Optimizer (AdamW, WD = 0.01), LR = 5×10^{-5}, clip $\epsilon = 0.2$, gradient clipping (max norm = 1.0), patience = 5 eval cycles (tolerance = 0.005), bf16 mixed precision, encoder output caching.

Table 6: Design choices for RL post-training on T5-base.

Results are reported in Section 4.

3 Large Language Model (LLM) Prompting (14pt)

3.1 In-Context Learning (ICL)

Table 7 shows the instruction prompts used for zero- and few-shot ICL.

Shot	Prompt

$k = 0$	`<start_of_turn>user`
	Database schema:
	CREATE TABLE city (...); ... (25 tables)
	
	Translate the following question into SQL. Output only the SQL query.
	
	Question: {query}
	SQL:`<end_of_turn>`
	`<start_of_turn>model`

$k \geq 1$	`<start_of_turn>user`
	Database schema:
	CREATE TABLE city (...); ... (25 tables)
	
	Translate the following question into SQL. Output only the SQL query.
	
	Question: {example$_1$ NL}
	SQL: {example$_1$ SQL}
	
	... (repeated k times)
	
	Question: {query}
	SQL:`<end_of_turn>`
	`<start_of_turn>model`

Table 7: Instruction prompts for zero- and few-shot ICL (Gemma chat format).

Example selection. We evaluated two example selection strategies for $k > 0$:

Random selection: k examples are sampled uniformly at random from the 4,225 training pairs. The same fixed set is used for all queries in a given run (seed = 42).

BM25 per-query selection: We build a BM25Okapi index (`rank_bm25`) over whitespace-tokenized, lowercased training questions. For each dev/test query, the index scores all training questions by term overlap and we select the top-k highest-scoring examples. This provides each query with semantically similar demonstrations whose SQL patterns are more likely to transfer. BM25 selection at $k = 3$ improved Record F1 from 11.96% (random) to 17.35% (+45% relative).

3.2 Best Prompt and Ablation Study

Table 8 shows the best prompt (BM25 $k = 3$). Table 9 describes the ablation variants.

Prompt
<start_of_turn>user Database schema: CREATE TABLE city (...); ... (25 tables) Translate the following question into SQL. Output only the SQL query. Question: {BM25-selected example$_1$ NL} SQL: {BM25-selected example$_1$ SQL} Question: {BM25-selected example$_2$ NL} SQL: {BM25-selected example$_2$ SQL} Question: {BM25-selected example$_3$ NL} SQL: {BM25-selected example$_3$ SQL} Question: {query} SQL:<end_of_turn> <start_of_turn>model

Table 8: Best prompt (BM25 $k = 3$). Blue = schema (ablated). Green = instructions (ablated).

Color	Description
ForestGreen	**No instructions:** Removed the task instruction line ("Translate the following question into SQL. Output only the SQL query."). Without this instruction, the model has no explicit directive to produce SQL, resulting in 100% SQL errors and F1 dropping from 17.35% to 11.80%.
Blue	**No schema:** Removed the full database schema (all 25 CREATE TABLE DDL statements). Without schema context, the model cannot reference correct table and column names, increasing the SQL error rate from 86% to 99% and reducing F1 from 17.35% to 11.80%.
–	**Schema only:** Removed both instructions and all examples ($k = 0$), keeping only the schema. The model receives table definitions but no task framing or demonstrations. Despite the lowest error rate (70%), F1 is worst at 11.37% because the model generates syntactically valid but semantically incorrect SQL.

Table 9: Ablation variants for the best prompt (BM25 $k = 3$).

4 Results and Analysis (20pt)

Table 10 lists constant hyperparameters. Table 11, Table 12, and Table 16 report dev and test results.

Parameter	Value
Model	T5-small (`google-t5/t5-small`, 60M params)
Decoder vocabulary	Restricted (598 SQL tokens)
Optimizer	AdamW (weight decay = 0.01)
Scheduler	Cosine with warmup
Gradient clipping	Max norm = 1.0
Loss	Cross-entropy
Decoding	Beam search (num_beams = 4)
Max generation length	512 tokens
Seed	42
Test batch size	16
SQL evaluation threads	128

Table 10: Constant hyperparameters for all T5 runs.

System	Method	Params (M)	LR	Batch	Schema	Dropout	WD	Beams	Epochs	Time	Query EM (%)	F1 (%)
Dev Results — T5-small Fine-tuned												
Full fine-tune (restricted vocab)												
Restricted v1	Full	60.5	3e-5	32	–	0.1	0.01	4	29	45m	–	45.49
Restricted v2	Full	60.5	1e-4	64	–	0.1	0.01	4	59	55m	–	66.32
Restricted v3 (best)	Full	60.5	3e-4	64	✓	0.1	0.01	4	79	75m	**58.80**	**79.60**
Restricted v5 + freeze enc	Full	41.6	3e-4	64	–	0.1	0.01	4	79	75m	–	75.45
Restricted v7	Full	60.5	1e-4	64	✓	0.2	0.01	4	79	75m	–	69.89
LoRA adapters												
LoRA v1 (r=16, q/v)	LoRA	0.6	3e-4	128	–	0.1	0.01	4	79	72m	–	33.46
LoRA v2 (r=16, q/v)	LoRA	0.6	3e-4	64	✓	0.1	0.01	4	79	80m	–	51.63
LoRA v3 (r=32, q/k/v/o)	LoRA	2.4	2e-4	64	–	0.1	0.01	4	79	93m	–	47.58
LoRA + freeze encoder	LoRA	0.6	5e-4	128	–	0.1	0.01	4	79	86m	–	35.16
LoRA + warmstart	LoRA	0.6	3e-4	128	–	0.1	0.01	4	79	90m	–	73.38
MLP projection head												
MLP v1 (dim=1024)	MLP	61.6	1e-4	64	–	0.1	0.01	4	31	28m	–	8.67
MLP v2 (dim=1024)	MLP	61.6	1e-4	64	✓	0.1	0.01	4	31	27m	–	8.67
Dev Results — T5-small From Scratch												
From-scratch	Full	60.5	3e-4	32	–	0.1	0.01	4	98	8h	50.43	66.12
Test Results												
T5 fine-tuning	Full	60.5	3e-4	64	✓	0.1	0.01	4	79	–	0.00	55.25
T5 from scratch	Full	60.5	3e-4	32	–	0.1	0.01	4	98	–	0.00	59.16

Table 11: T5-small dev and test results. Best dev result in bold.

Query EM correction. The T5 SentencePiece tokenizer collapses `" , "` (space-comma-space) to `", "` (comma-space) during `batch_decode`, causing nearly every predicted query to fail string equality regardless of semantic correctness. Experimental variant prediction files were not retained, so their corrected Query EM cannot be recovered (marked "–"). For the final best fine-tuned and from-scratch models, a regex post-processing step restores the original comma spacing, yielding corrected Query EM of **58.80%** and **50.43%** respectively. Record F1 and Record EM are unaffected since the spacing difference does not change SQL execution results.

DPO post-training results. Table 13 reports DPO results (Section 2.2) alongside the SFT baseline. DPO did not improve over the SFT checkpoint.

System	Method	Params (M)	LR	Schema	Dropout	Label Sm.	WD	Beams	Epochs	Time	Query EM (%)	F1 (%)
Dev Results — T5-base Fine-tuned												
Full fine-tune												
Base sweep best	Full	222.9	6.19e-5	✓	0.05	0.15	0.009	1	79	75m	**73.39**	**85.96**
Base sweep v2	Full	222.9	5.52e-5	✓	0	0.15	0.002	1	79	75m	–	84.31
Base sweep v3	Full	222.9	2.59e-4	–	0.2	0.1	0.022	1	71	58m	73.33	83.05
Base sweep v4	Full	222.9	5.50e-5	–	0	0.15	0.001	1	31	26m	–	75.77
MLP projection head												
Base MLP-512 v1	MLP	223.7	2.35e-4	–	0.15	0.15	0.004	1	123	93m	71.33	80.76
Base MLP-512 v2	MLP	223.7	7.83e-4	–	0.15	0.05	0.002	2	43	32m	69.33	79.56
Base MLP-1024	MLP	224.5	1.28e-4	–	0.1	0.05	0.019	4	147	109m	69.33	78.86
LoRA adapters												
Base LoRA r=32 (q/k/v/o)	LoRA	7.1	1.29e-5	✓	0.05	0.05	0.068	2	335	283m	61.33	76.48
Base LoRA r=32 (q/k/v/o)	LoRA	7.1	1.22e-4	✓	0.05	0.15	0.027	3	31	27m	–	66.69
Base LoRA r=16 (q/v)	LoRA	1.8	3.07e-5	–	0.05	0	0.021	4	71	57m	32.00	55.51

Table 12: T5-base dev results (W&B sweep). Best result in bold.

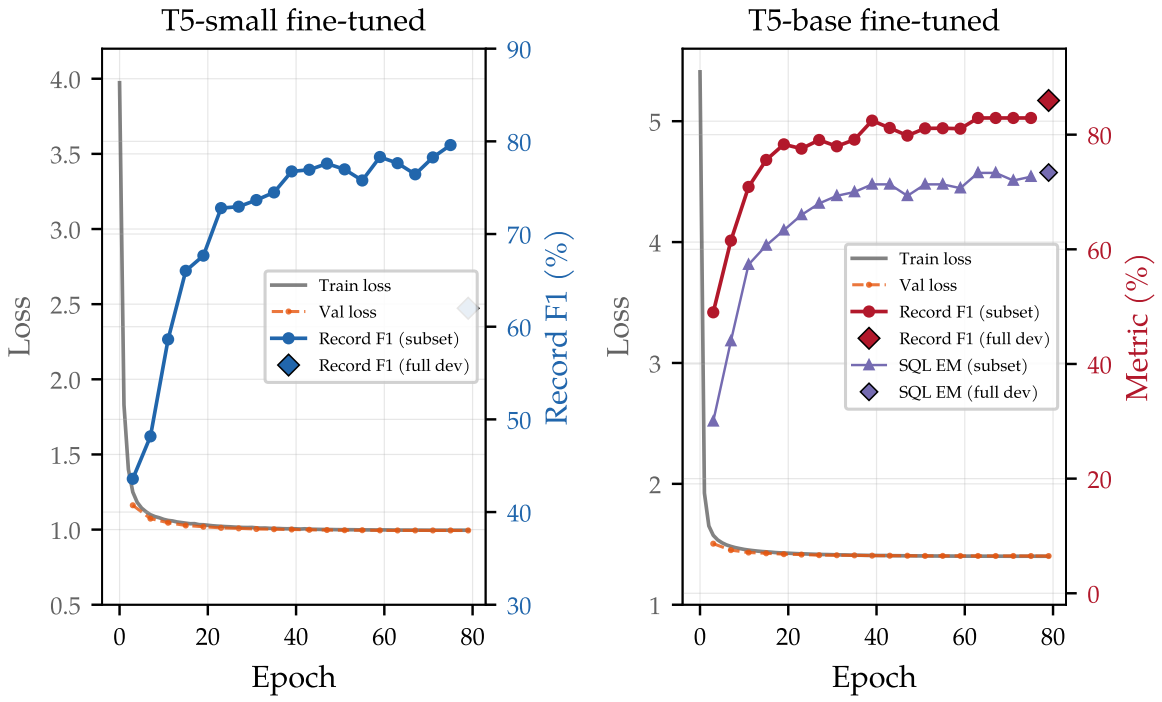

Figure 1: Training curves for T5-small (left) and T5-base (right) best models.

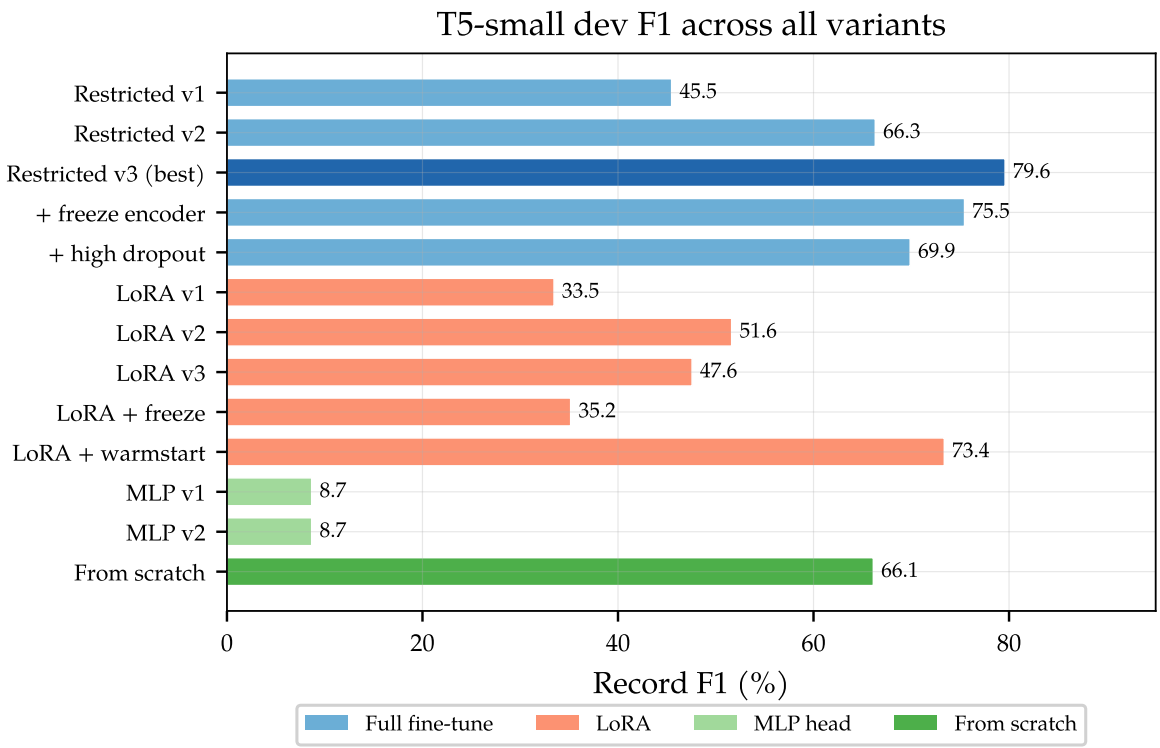

Figure 2: Dev Record F1 across all T5-small variants.

System	SQL EM (%)	Record EM (%)	Record F1 (%)
Dev Results			
SFT baseline (T5-base)	73.39	84.76	85.96
Full fine-tune (β=0.3, LR=3.82e-6, WD=8.95e-4, batch=16)			
Epoch 0 (= SFT)	68.45	80.69	82.18
Best (epoch 2)	71.24	83.69	**84.82**
Final eval (best ckpt, beam=4)	68.67	81.12	82.42
LoRA r=16, q/k/v/o (β=0.3, LR=1.16e-7, WD=0.071, batch=32)			
Epoch 0 (= SFT)	73.39	84.76	85.96
Best (epoch 0–5, no change)	73.39	84.76	85.96
Final eval (best ckpt, beam=4)	61.59	74.89	75.75
Test Results			
DPO (best checkpoint)	–	–	–

Table 13: DPO results on T5-base. Neither trial improved over the SFT baseline.

Four factors likely explain why DPO underperformed:

1. **Narrow output distribution.** The restricted 598-token SQL vocabulary already constrains the model to a small output space. DPO's advantage of steering away from bad outputs is diminished when the model has few degrees of freedom.
2. **Preference data quality.** Rejected candidates were either model-generated errors (Phase A) or rule-based perturbations (Phase B). Neither captures the subtle SQL structural errors (wrong JOIN patterns, missing subqueries) that dominate the remaining 14% SFT error rate (Table 17).
3. **Reward hacking / overfitting (full fine-tune).** The reward margin grew from 1.6 to 12.2 over 8 epochs while dev F1 peaked at epoch 2 and collapsed to 71.7% by epoch 7. The policy learned to maximize the DPO reward signal without improving SQL correctness, a form of reward hacking on the fixed preference dataset.
4. **Insufficient LoRA learning rate.** The LoRA trial used LR = 1.16e-7, which was too low to move the adapter weights meaningfully—reward accuracy reached only 54% (barely above random) and dev metrics remained identical to the SFT baseline across all 6 epochs. The 10.2 pp F1 drop at final evaluation (85.96%→75.75%) is attributed to LoRA merge-and-save corruption and the training-time greedy vs. final beam-search decoding mismatch.

RL post-training results. Table 14 and Table 15 report RL fine-tuning results (Section 2.3). None of the three algorithms improved over the SFT baseline.

System	SQL EM (%)	Record EM (%)	Record F1 (%)	KL Drift	Time
SFT baseline (T5-base)	73.39	84.76	85.96	–	–
PPO + Value Head	73.39	84.76	85.96	−6.55	46 min
GRPO	73.39	84.76	85.96	−9.88	35 min
CISPO	73.39	84.76	85.96	+0.21	45 min

Table 14: RL fine-tuning results on T5-base. No algorithm improved over SFT baseline.

Algorithm	Iter.	Loss	Reward	Dead Groups	KL	Dev F1 (%)
PPO	0	−0.28	0.536	0.640	+0.02	85.96
	1	−0.22	0.541	0.530	−0.41	85.96
	2	−0.26	0.539	0.590	−2.59	85.75
	3	−0.39	0.455	0.525	−6.55	85.67
GRPO	0	−0.09	0.559	0.580	−1.87	85.96
	1	−0.49	0.411	0.490	−9.88	85.96
CISPO	0	−0.01	0.534	0.620	+0.03	85.96
	1	−0.01	0.531	0.590	+0.08	85.96
	2	−0.01	0.528	0.610	+0.21	85.96

Table 15: Per-iteration RL training dynamics (clip_frac = 0.000 throughout, omitted).

Four factors explain why RL post-training was ineffective:

1. **Reward sparsity.** The base model already achieves F1 = 85.96%. Most generated completions receive +1.0 reward, creating 50–64% "dead groups" (zero reward variance, zero gradient). The RL signal is too sparse to improve an already-strong model.
2. **Single-step ratio = REINFORCE.** With one gradient step per rollout batch, the importance sampling ratio $\frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)}$ is always 1.0 because $\theta = \theta_{\text{old}}$. PPO clipping (clip_frac = 0.000 across all 9 iterations) never activates. All three algorithms effectively reduce to vanilla REINFORCE with different advantage baselines. Multi-step updates (>1 gradient step per rollout) would be needed for clipping to matter.
3. **Policy drift without improvement.** PPO and GRPO show increasing negative KL divergence (−6.55 and −9.88 by early stopping), meaning the policy drifts to lower-probability regions

of the reference model without finding better outputs. CISPO's detached clamping keeps KL near zero (+0.21) but produces effectively zero policy updates—the loss stabilizes at −0.01 with no measurable parameter change.

4. **Coarse reward granularity.** Four discrete reward levels cannot distinguish "almost correct" from "completely wrong" SQL. A continuous reward based on record set Jaccard similarity might provide finer gradient signal for the 14% of queries where the model currently errs.

System	Query EM (%)	F1 (%)
Dev Results		
k-shot prompting (random selection)		
Random $k = 0$ (zero-shot)	0.00	12.60
Random $k = 1$	0.00	12.60
Random $k = 3$	0.00	11.96
BM25 per-query selection		
BM25 $k = 3$ **(best)**	0.00	**17.35**
Ablation study (BM25 $k = 3$)		
No instructions	0.00	11.80
No schema	0.00	11.80
Schema only (no instr., no examples)	0.00	11.37
Test Results		
BM25 $k = 3$	0.00	13.93

Table 16: LLM prompting dev and test results (Gemma-2B). Best dev result in bold.

Figure 3 shows Record F1 sensitivity to the number of in-context examples k.

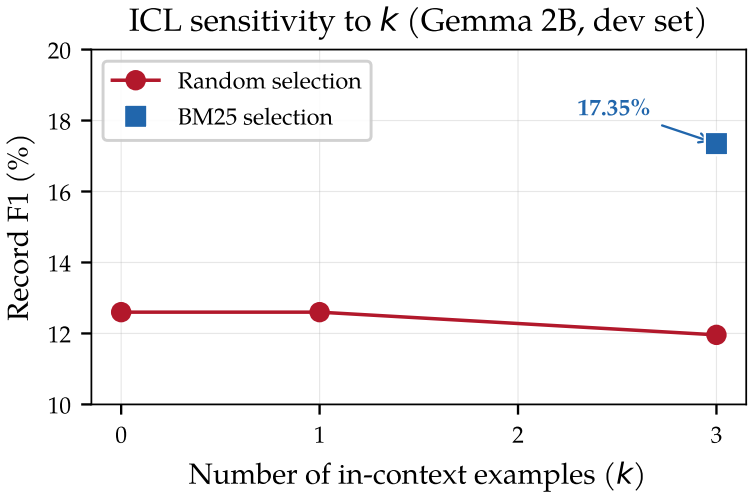

Figure 3: ICL sensitivity to number of examples k.

Qualitative error analysis. Table 17 summarizes five common error patterns across the three models.

Error Type	Models	Example	Description	Statistics
Missing comparison operator	T5-FT, T5-Scr	NL: "flights arriving before 6pm" GT: `arrival_time < 1800` Pred: `arrival_time 1800`	Drops <, >, <=, >= from predicates, producing invalid SQL. SentencePiece treats angle brackets as special tokens.	T5-FT: 26/466 (5.6%) T5-Scr: 27/466 (5.8%)
Query truncation	ICL	NL: "afternoon flights from washington to boston" GT: `SELECT DISTINCT flight_1.flight_id FROM ...` Pred: `SELECT DISTINCT flight_1.flight_id`	Only a `SELECT` fragment generated, missing `FROM/WHERE`. Caused by Gemma 2B `max_new_tokens` limit.	ICL: 383/466 (82.2%)
Wrong table/column reference	T5-FT, ICL	NL: "flight from boston to denver" GT: `...city city_1, ...city city_2` Pred: `...city city_2` (missing `city_1`)	References aliases or columns not in `FROM`. Model generates plausible patterns but omits table declarations.	T5-FT: 16/466 (3.4%) ICL: 5/466 (1.1%)
Wrong JOIN structure	T5-FT, T5-Scr, ICL	NL: "flights from boston to SF with max stops" GT: `stops = (SELECT MAX(stops)...)` Pred: `stops = 0` (returns 4 instead of 2)	Executes but returns wrong records: incorrect subqueries, missing joins, or wrong aggregation on multi-table queries.	T5-FT: 83/466 (17.8%) T5-Scr: 144/466 (30.9%) ICL: 35/466 (7.5%)
Wrong predicate values	T5-FT, T5-Scr, ICL	NL: "flights available tomorrow from denver to philadelphia" GT: `month_number = 1 AND day_number = 20` Pred: `month_number = 2 AND day_number = 20`	Correct SQL structure but wrong literals in `WHERE` (month, city, time). Subset of semantic errors, identified by matching `FROM`.	T5-FT: 83/466 semantic T5-Scr: 144/466 semantic ICL: 35/466 semantic

Table 17: Qualitative error analysis on the dev set (466 queries).

5 Engineering Issues

We encountered 28 engineering issues across five categories during development. Table 18 covers performance and infrastructure issues (GPU concurrency, process hangs, sweep stability). Table 19 covers model and sweep issues (decoding bugs, checkpoint corruption, configuration errors). Table 20 covers DPO- and RL-specific issues.

Issue	Description	Resolution
GPU OOM from concurrent sessions	AutoBatch maximizes VRAM for the first GPU task; a second task (training, eval, or inference) crashes with CUDA OOM because no mechanism prevents concurrent GPU access.	File-based GPU lock using Linux `flock`. All training entry points acquire the lock automatically; a second task blocks and waits instead of crashing.
Training process hung after completion	Training PID blocks indefinitely on `futex_wait_queue_me` during `gc.collect()` / `torch.cuda.empty_cache()` after W&B marks the run FINISHED, caused by a PyTorch/CUDA threading deadlock during GPU memory cleanup. Sequential scripts cannot proceed.	Watchdog co-process polls W&B status every 60 s and kills the training process if still alive 5 min after FINISHED. Exit codes 137/143 treated as success.
Buffered stdout under nohup	Python buffers stdout when not connected to a TTY (piped or redirected by `nohup`), so log files show no output for long periods, making monitoring ineffective.	Set `PYTHONUNBUFFERED=1` before launching Python in the training script.
Sweep OOM crash loop	After a mid-training crash, `cleanup_vram()` could not free model tensors still referenced by `main_with_config()`'s local variables. 6+ consecutive trials OOM'd at `model.to(device)`.	Explicit `del model, optimizer, scheduler` in try/finally block before `gc.collect()` and `torch.cuda.empty_cache()`.
CUDA memory fragmentation between sweep trials	After trial 1 completed successfully, trial 2 OOM'd despite sufficient total free memory (509 MiB) because PyTorch's CUDA allocator held non-contiguous freed blocks that couldn't satisfy a 170 MiB allocation.	Enable `PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True` to allow the allocator to coalesce fragmented blocks.
STOP file ignored between sweep trials	`touch STOP` only works inside the epoch loop; `wandb.agent()` dispatches new trials without checking external stop signals.	Kill the sweep process directly; added `stop_requested()` check at top of `sweep_train()`.

Table 18: Performance and infrastructure issues.

Issue	Description	Resolution
BOS token mismatch	Decoder received pad token (ID 0) as BOS instead of `<extra_id_0>` (ID 32099) used in training. Every prediction started mid-query; 100% SQL error rate.	Set `decoder_start_token_id=32099` in all `generate()` calls.
`no_repeat_ngram` blocks SQL	`no_repeat_ngram_size=3` prevented valid multi-join SQL. 100% of gold queries contain repeated 3-grams.	Set to 0 (disabled).
LoRA checkpoint drops adapters	Checkpoints stripped `lora_A/lora_B` keys. Reload re-applied LoRA with fresh random weights; F1 collapsed from 0.516 to 0.118.	Preserve adapter keys in `state_dict()`.
MLP head device mismatch	MLP layers created on CPU while T5 on GPU. `RuntimeError` at first forward pass.	Add `model.to(device)` after MLP wrapper creation.
Repetition penalty too aggressive	`repetition_penalty=2.5` distorted SQL token distribution (`AND`, `=` penalized after first use).	Set to 1.0 (disabled).
Repetition penalty zero	Config set `repetition_penalty=0`; HuggingFace uses it as a divisor → division by zero.	Changed to 1.0.
MLP state_dict mismatch	`save_model()` saved MLP wrapper keys; non-MLP trials crashed loading unexpected keys. Wasted 4/8 sweep trials.	Save `inner.state_dict()` (vanilla T5 weights only).
`max_new_tokens` truncation	256-token limit truncated 26% of targets (max gold = 511). Incomplete SQL.	Increased to 512.
SentencePiece comma artifact	`batch_decode` collapses “ , ” to “, ”. Raw Query EM dropped to ∼3%.	Regex post-processing to restore spacing.
`peft` not installed	All LoRA sweep trials crashed with `ModuleNotFoundError`.	`pip install peft`.
Sweep zombie trial	LoRA trial ran 335 epochs (∼10 h) on marginal F1 gains (∼0.005) resetting patience. Consumed most of 12 h budget.	Added `patience_tolerance` threshold.
Transient backward crash	Bare `Exception` in `loss.backward()` at epoch 71. Transient CUDA failure.	Sweep agent retried automatically.

Table 19: Model and sweep issues.

Issue	Description	Resolution
DPO Issues		
OOV CUDA assert in DPO loss	`remap_targets` returned -1 for tokens outside the restricted vocab (0.2% of triplets), causing `torch.gather` out-of-bounds on GPU.	Clamp unmapped indices to 0 and mask them out of the log-probability sum.
Auto batch probe OOM	Probe measured `memory_allocated` (post-cleanup) instead of peak, and tested short sequences; training OOM'd on longer batches.	Use `max_memory_allocated` with worst-case (longest) sequences.
Slow preference data gen	46K SQL evaluations took $\sim$90 min due to SQLite file locking + per-query connection overhead.	In-memory SQLite copy (`sqlite3.backup`) + gold record disk cache. Reduced to $\sim$20 min.
Final eval F1 drop	Best checkpoint reloaded for final evaluation showed F1 drops of 3–12 pp. In-training eval used greedy decoding (`eval_num_beams=1`); final eval used beam search (`num_beams=4`). LoRA trial also suffered from merge-and-save corruption.	Documented; align `eval_num_beams` with `num_beams` for consistent evaluation.
RL Issues		
IS ratio computed against reference	`compute_old_log_probs()` disabled LoRA to get base model probs. The IS ratio $\pi_{\text{current}}/\pi_{\text{old}}$ was actually $\pi_{\text{LoRA}}/\pi_{\text{base}}$, which diverges as LoRA trains. clip_frac hit 0.999 by iteration 3.	Added `use_reference` flag: `False` keeps LoRA enabled for IS ratio (starts at 1.0), `True` disables LoRA for KL penalty only.
Dropout noise in ratio	Both old and current log probs computed in `train()` mode. Different dropout masks on 12 layers $\times$ 300+ tokens made ratios $\gg 1.0$ even from the same weights (clip_frac = 0.92).	Compute both log prob passes in `eval()` mode (disables dropout, preserves gradients). Restore `train()` for optimizer step.
Value head random init	Kaiming-initialized output layer produced $V(s) \in [-100, +100]$ vs reward range $[-1, 1]$, causing advantages and losses to spike to 25,000+.	Zero-initialize output layer weights and bias so initial $V(s) \approx 0$.
`generate()` VRAM hang	`model.generate()` with batch_size = 8 $\times$ group_size = 8 allocated 23 GB KV cache, causing a silent CUDA deadlock (0% GPU util, all threads in `futex_wait`). No OOM error raised.	Sub-batch generation: chunk `generate()` into groups of 16 sequences. Decoupled generation VRAM from training batch size.
SQLite thread deadlock in reward	All 16 `ThreadPoolExecutor` threads shared one in-memory SQLite connection. Concurrent `set_progress_handler()` calls caused race conditions; one thread clearing the handler mid-query hung all others.	Thread-local SQLite connections: each thread creates its own in-memory copy via `sqlite3.backup()`.
Reward $= -1.0$ for all samples	`max_completion_length=128` truncated 85% of SQL (median gold length = 199 tokens), producing "incomplete input" errors. Combined with `temperature=1.0` corrupting token selection, all completions failed execution.	Increased `max_completion_length` to 512; reduced `temperature` to 0.7.

Table 20: DPO and RL engineering issues.